

Capstone Agentic Testing v1.5

February 2, 2026

1 imports and housekeeping

```
[1]: import requests
import csv
import time
import os
import re
import json
from pathlib import Path
from datetime import datetime
from typing import Dict, Any, List
```

```
[2]: # local parameters
API_URL = "http://localhost:1234/v1/chat/completions"
MODEL = "deepseek-coder-6.7b-instruct"

DIR_ROOT = Path.cwd()
OUTPUT_DIR = DIR_ROOT / "automated datasets"
PROMPT_DIR = DIR_ROOT / "automated datasets" / "prompt txt"
```

```
[3]: # knobs you will edit
N_ITEMS = 5

GEN_TEMPERATURE = 0.4
GEN_MAX_TOKENS = 250

REV_TEMPERATURE = 0.0
REV_MAX_TOKENS = 200

SLEEP_SECONDS = 0.5
REQUEST_TIMEOUT_SECONDS = 600
```

```
[4]: # file output setup
os.makedirs(OUTPUT_DIR, exist_ok=True)

RUN_ID = datetime.now().strftime("%Y%m%d_%H%M%S")
```

```

# raw "log" CSVs
GEN_RAW_CSV = os.path.join(OUTPUT_DIR, f"generator_output_{RUN_ID}.csv")
REV_RAW_CSV = os.path.join(OUTPUT_DIR, f"review_decisions_{RUN_ID}.csv")

# clean "analysis-ready" CSVs
ITEMS_CLEAN_CSV = os.path.join(OUTPUT_DIR, f"items_clean_{RUN_ID}.csv")
REVIEWS_CLEAN_CSV = os.path.join(OUTPUT_DIR, f"reviews_clean_{RUN_ID}.csv")
ANALYSIS_MERGED_CSV = os.path.join(OUTPUT_DIR, f"analysis_merged_{RUN_ID}.csv")

```

2 functions

2.1 import and housekeeping

```

[5]: def call_llm(system_prompt: str, user_prompt: str, temperature: float,
    ↪max_tokens: int) -> str:
    """
    Sends a single chat-completion request to a locally hosted LLM
    via LM Studio's OpenAI-compatible API.

    - `system_prompt` defines the agent role and behavioral constraints.
    - `user_prompt` supplies the task-specific content.
    - `temperature` controls output variability.
    - `max_tokens` caps response length.

    Returns the raw text content of the model's response.
    No parsing, validation, or error recovery is performed here.
    """
    payload = {
        "model": MODEL,
        "messages": [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        "temperature": temperature,
        "max_tokens": max_tokens
    }
    r = requests.post(API_URL, json=payload, timeout=REQUEST_TIMEOUT_SECONDS)
    r.raise_for_status()
    return r.json()["choices"][0]["message"]["content"]

```

```

[6]: def write_csv_header_if_needed(path: str, header: List[str]) -> None:
    """
    Ensures a CSV file has a header row before appending data.

    If the file does not exist or is empty, writes the provided
    header row. If the file already contains data, no action
    is taken.
    """

```

This allows safe reuse of the same output files across multiple batch runs without duplicating headers.

```
"""
if not os.path.exists(path) or os.path.getsize(path) == 0:
    with open(path, "w", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow(header)
```

```
[7]: def clean_generator_text(text: str) -> str:
    """
    Keep only the required fields starting at 'Item ID:'.
    Drops preamble like 'Sure...' and trailing notes/snippets.

    This does not guarantee perfect format compliance; it only
    removes obvious non-item boilerplate so downstream parsing
    has a fair chance.
    """
    m = re.search(r"Item ID:.*", text, flags=re.DOTALL)
    return m.group(0).strip() if m else text.strip()
```

```
[8]: def extract_first_json_obj(text: str) -> Dict[str, Any]:
    """
    Extract the first JSON object from a response.

    - Strips known local-model junk tokens that sometimes appear.
    - Finds the first {...} block and attempts json.loads().

    Raises ValueError if no JSON object can be extracted.
    """
    text = text.replace("<begin of sentence>", "").replace("\u0000", "")
    m = re.search(r"\{.*\}", text, flags=re.DOTALL)
    if not m:
        raise ValueError("No JSON object found in reviewer output.")
    return json.loads(m.group(0))
```

```
[9]: def normalize_decision(item_id: str, raw_obj: Dict[str, Any]) -> Dict[str, Any]:
    """
    Enforces a stable reviewer decision schema so metrics and joins
    are reliable even when the reviewer drifts.

    - Overwrites `item_id` with the pipeline item_id (source of truth).
    - Fills missing keys with defaults.
    - Clamps enums to ACCEPT/REVISE/REJECT and HARD/REPAIRABLE/NONE.
    - Enforces decision failure_layer consistency for analysis.
    """
    out = {
        "item_id": item_id,
```

```

        "decision": "REJECT",
        "failure_layer": "HARD",
        "reason_codes": [],
        "revision_instructions": []
    }

    if isinstance(raw_obj, dict):
        # tolerate minor drift
        if "decision" in raw_obj:
            out["decision"] = str(raw_obj["decision"]).strip().upper()
        elif "Decision" in raw_obj:
            out["decision"] = str(raw_obj["Decision"]).strip().upper()

        if "failure_layer" in raw_obj:
            out["failure_layer"] = str(raw_obj["failure_layer"]).strip().upper()
        elif "FailureLayer" in raw_obj:
            out["failure_layer"] = str(raw_obj["FailureLayer"]).strip().upper()

        if "reason_codes" in raw_obj and isinstance(raw_obj["reason_codes"],
↪list):
            out["reason_codes"] = raw_obj["reason_codes"]
        elif "reasonCodes" in raw_obj and isinstance(raw_obj.
↪get("reasonCodes"), list):
            out["reason_codes"] = raw_obj["reasonCodes"]

        if "revision_instructions" in raw_obj and
↪isinstance(raw_obj["revision_instructions"], list):
            out["revision_instructions"] = raw_obj["revision_instructions"]
        elif "revisionInstructions" in raw_obj and isinstance(raw_obj.
↪get("revisionInstructions"), list):
            out["revision_instructions"] = raw_obj["revisionInstructions"]

        # clamp allowed enums
        if out["decision"] not in {"ACCEPT", "REVISE", "REJECT"}:
            out["decision"] = "REJECT"
            out["failure_layer"] = "HARD"
            out["reason_codes"] = list(set(out["reason_codes"] +
↪["REVIEWER_INVALID_DECISION"]))

        if out["failure_layer"] not in {"HARD", "REPAIRABLE", "NONE"}:
            out["failure_layer"] = "HARD"
            out["reason_codes"] = list(set(out["reason_codes"] +
↪["REVIEWER_INVALID_LAYER"]))

        # ensure lists
        if not isinstance(out["reason_codes"], list):

```

```

        out["reason_codes"] = ["REVIEWER_REASON_CODES_NOT_LIST"]
    if not isinstance(out["revision_instructions"], list):
        out["revision_instructions"] = ["REVIEWER_REV_INSTRUCTIONS_NOT_LIST"]

    # enforce decision failure_layer consistency for analytics
    if out["decision"] == "ACCEPT":
        out["failure_layer"] = "NONE"

    if out["decision"] in {"REVISE", "REJECT"} and out["failure_layer"] == \
↪ "NONE":
        # conservative default; you can refine later using reason_codes
        out["failure_layer"] = "HARD"

    return out

```

```

[10]: def _grab_field(text: str, label: str) -> str:
        """
        Extracts the content for a label like 'Question:' up to the next known \
↪ label.
        Returns empty string if not found.
        """
        # normalize line endings for more stable matching
        t = text.replace("\r\n", "\n")
        # known next-field labels
        next_labels = [
            "Item ID:", "Question:", "A)", "B)", "C)", "D)",
            "Correct Answer:", "Rationale:", "Objective Tag:", "Difficulty:"
        ]

        # build a regex that captures label content until the next label
        next_pat = "|".join(re.escape(x) for x in next_labels if x != label)
        pat = rf"{re.escape(label)}\s*(.*?)\s*(?={next_pat}|$)"
        m = re.search(pat, t, flags=re.DOTALL)
        return m.group(1).strip() if m else ""

```

```

[11]: def validate_generator_schema(gen_text: str) -> tuple[bool, list[str]]:
        """
        Performs a lightweight schema/format gate on generator output BEFORE review.
        Intended to catch contract violations early (e.g., missing labels, code \
↪ fences,
        invalid correct_key, invalid difficulty) and avoid wasting reviewer \
↪ tokens.
        Returns: (is_valid, violations) where violations is a list of short codes.
        """
        t = gen_text or ""
        violations: list[str] = []

```

```

# required labels (contract fields)
required_labels = ["Item ID:", "Question:", "A)", "B)", "C)", "D)",
↳ "Correct Answer:", "Difficulty:"]
for lab in required_labels:
    if lab not in t:
        violations.append(f"MISSING_{lab.replace(' ', '').replace(':', '')}.
↳ replace(' ', '')}")

# code fence / code artifact detection
if "`" in t or "print(" in t or "def " in t or "import " in t:
    violations.append("CODE_ARTIFACT")

# correct key must parse to A-D
ck = extract_correct_key(t)
if ck is None or ck not in {"A", "B", "C", "D"}:
    violations.append("BAD_CORRECT_KEY")

# difficulty must parse to Recall/Application
diff = extract_difficulty_tag(t)
if diff is None or diff not in {"Recall", "Application"}:
    violations.append("BAD_DIFFICULTY")

# must end at Difficulty line (hard trim should enforce this; if not, flag)
trimmed = hard_trim_after_difficulty(t)
if trimmed != t:
    violations.append("TRAILING_AFTER_DIFFICULTY")

return (len(violations) == 0, violations)

```

2.2 parsing and cleaning

```

[12]: # configuration / tools
_RX_CORRECT = re.compile(r"(?im)^\s*Correct\s*Answer\s*:\s*([ABCD])\b")
_RX_DIFF = re.compile(r"(?im)^\s*Difficulty\s*:\s*(Recall|Application)\b")
_RX_DIFF_LINE = re.compile(r"(?im)^\s*Difficulty\s*:\s*.*?$")

```

```

[13]: def extract_correct_key(text: str) -> str | None:
    """
    Extracts the canonical correct answer key (A, B, C, or D).
    Captures only the first valid letter following 'Correct Answer:'.
    Safely ignores any trailing text such as ' ) Option text' or explanations.
    Returns None if no valid key is found.
    """
    m = _RX_CORRECT.search(text or "")
    return m.group(1) if m else None

```

```
[14]: def extract_difficulty_tag(text: str) -> str | None:
    """
    Extracts the difficulty classification used for analysis.
    Accepts only the first valid token: 'Recall' or 'Application'.
    Discards any additional narrative text after the difficulty label.
    Returns None if the difficulty line is missing or malformed.
    """
    m = _RX_DIFF.search(text or "")
    return m.group(1) if m else None
```

```
[15]: def hard_trim_after_difficulty(text: str) -> str:
    """
    Enforces the generator output contract by trimming any text that appears
    ↪after the 'Difficulty:' line.
    Prevents leakage such as explanations, code blocks, or commentary that
    ↪violate the expected item format.
    If no difficulty line is found, returns the text unchanged.
    """
    if not text:
        return ""
    m = _RX_DIFF_LINE.search(text)
    if not m:
        return text.strip()
    return text[:m.end()].strip()
```

```
[16]: def parse_generator_item(gen_text: str) -> Dict[str, Any]:
    """
    Parses a generator blob into structured fields for analysis.

    This is a best-effort parser:
    - It assumes the model mostly follows the template.
    - It will return parse_ok=False if core fields are missing.

    Output fields are intended for CSV columns and prompt iteration,
    not for production item banking.
    """
    t = clean_generator_text(gen_text)

    item_id_field = _grab_field(t, "Item ID:")
    question = _grab_field(t, "Question:")
    option_a = _grab_field(t, "A")
    option_b = _grab_field(t, "B")
    option_c = _grab_field(t, "C")
    option_d = _grab_field(t, "D")
    correct = _grab_field(t, "Correct Answer:")
    rationale = _grab_field(t, "Rationale:")
    objective = _grab_field(t, "Objective Tag:")
```

```

difficulty = _grab_field(t, "Difficulty:")

# normalize correct key to A/B/C/D if possible
correct_key = ""
if correct:
    c = correct.strip()
    # allow "A", "A)", "A) blah", "A: blah"
    m = re.match(r"^s*([ABCD])b", c, flags=re.IGNORECASE)
    if m:
        correct_key = m.group(1).upper()
    else:
        m = re.match(r"^s*([ABCD])\)", c, flags=re.IGNORECASE)
        correct_key = m.group(1).upper() if m else ""

# parse_ok heuristic: must have question, 4 options, and a correct key
parse_ok = bool(question and option_a and option_b and option_c and
↪option_d and correct_key)

return {
    "item_id_field": item_id_field,
    "question": question,
    "option_a": option_a,
    "option_b": option_b,
    "option_c": option_c,
    "option_d": option_d,
    "correct_key": correct_key,
    "rationale": rationale,
    "objective_tag": objective,
    "difficulty_tag": difficulty,
    "gen_parse_ok": parse_ok,
    "gen_raw": gen_text
}

```

```

[17]: def enforce_hygiene_on_review(gen_text: str, review_norm: dict) -> dict:
    """
    Deterministically enforces hygiene rules regardless of what the reviewer_
↪model returns.

    If hygiene fails, overrides decision to REVISE with FORMAT_VIOLATION.
    Keeps pipeline stable: every item gets a consistent, schema-valid review_
↪record.
    """
    t = gen_text or ""
    violations: list[str] = []

    # correct Answer must be exactly A/B/C/D (canonical parse must succeed)
    ck = extract_correct_key(t)
    if ck not in {"A", "B", "C", "D"}:

```



```

        violations.append("BAD_CORRECT_KEY")

        # difficulty must be exactly Recall/Application (canonical parse must
        ↳succeed)
        diff = extract_difficulty_tag(t)
        if diff not in {"Recall", "Application"}:
            violations.append("BAD_DIFFICULTY")

        # no code artifacts anywhere in options (or item)
        if "`" in t or "print(" in t or "def " in t or "import " in t:
            violations.append("CODE_ARTIFACT")

        # no trailing text after Difficulty line (hard trim should make this true;
        ↳if not, flag)
        trimmed = hard_trim_after_difficulty(t)
        if trimmed != t:
            violations.append("TRAILING_AFTER_DIFFICULTY")

        if not violations:
            return review_norm

        # hygiene failed: override to deterministic REVISE
        return {
            "item_id": review_norm.get("item_id"),
            "decision": "REVISE",
            "failure_layer": "SOFT",
            "reason_codes": ["FORMAT_VIOLATION"],
            "revision_instructions": [
                f"Hygiene gate failed: {'', '.join(violations)}". "
                "Fix formatting strictly: Correct Answer must be a single letter
        ↳A-D; "
                "Difficulty must be exactly Recall or Application; remove code
        ↳artifacts; "
                "end the item at the Difficulty line."
            ]
        }
    }

```

```

[18]: def post_run_cleanup(run_id: str) -> None:
        """
        Reads the raw output CSVs for this run and emits analysis-ready CSVs:
        - items_clean_<RUN_ID>.csv
        - reviews_clean_<RUN_ID>.csv
        - analysis_merged_<RUN_ID>.csv (joined on run_id + item_id)

        This is the bridge from "LLM blobs" to "iterable prompt engineering data".
        """
        # load generator raw rows

```

```

gen_rows = []
with open(GEN_RAW_CSV, "r", encoding="utf-8", newline="") as f:
    reader = csv.DictReader(f)
    for r in reader:
        if r.get("run_id") != run_id:
            continue
        gen_rows.append(r)

# parse generator items
items_clean = []
for r in gen_rows:
    item_id = r["item_id"]
    gen_output = r["gen_output"]
    parsed = parse_generator_item(gen_output)
    items_clean.append({
        "run_id": run_id,
        "item_id": item_id,
        **{k: v for k, v in parsed.items() if k != "gen_raw"},
        "gen_raw": parsed["gen_raw"]
    })

# load reviewer raw rows
rev_rows = []
with open(REV_RAW_CSV, "r", encoding="utf-8", newline="") as f:
    reader = csv.DictReader(f)
    for r in reader:
        if r.get("run_id") != run_id:
            continue
        rev_rows.append(r)

# parse reviewer decisions
reviews_clean = []
for r in rev_rows:
    item_id = r["item_id"]
    raw_json_text = r["review_json"]

    reviewer_parse_ok = True
    try:
        obj = json.loads(raw_json_text)
    except Exception:
        reviewer_parse_ok = False
        obj = {
            "item_id": item_id,
            "decision": "REJECT",
            "failure_layer": "HARD",
            "reason_codes": ["REVIEWER_PARSE_FAIL"],
            "revision_instructions": []

```

```

    }

    # normalize (schema + enums)
    norm = normalize_decision(item_id, obj)

    reviews_clean.append({
        "run_id": run_id,
        "item_id": item_id,
        "decision": norm["decision"],
        "failure_layer": norm["failure_layer"],
        "reason_codes": json.dumps(norm["reason_codes"]),
↪ensure_ascii=False),
        "revision_instructions": json.dumps(norm["revision_instructions"]),
↪ensure_ascii=False),
        "reviewer_parse_ok": reviewer_parse_ok,
        "review_raw_json": raw_json_text
    })

# write items_clean CSV
items_header = [
    "run_id", "item_id",
    "item_id_field", "question",
    "option_a", "option_b", "option_c", "option_d",
    "correct_key", "rationale", "objective_tag", "difficulty_tag",
    "gen_parse_ok", "gen_raw"
]
with open(ITEMS_CLEAN_CSV, "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=items_header)
    w.writeheader()
    for row in items_clean:
        w.writerow(row)

# write reviews_clean CSV
reviews_header = [
    "run_id", "item_id",
    "decision", "failure_layer",
    "reason_codes", "revision_instructions",
    "reviewer_parse_ok", "review_raw_json"
]
with open(REVIEWS_CLEAN_CSV, "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=reviews_header)
    w.writeheader()
    for row in reviews_clean:
        w.writerow(row)

# merge for analysis CSV
rev_by_item = {(r["run_id"], r["item_id"]): r for r in reviews_clean}

```

```

merged_rows = []
for it in items_clean:
    key = (it["run_id"], it["item_id"])
    rev = rev_by_item.get(key, {})
    merged = {**it}
    # prefix reviewer columns to avoid collisions
    merged.update({
        "rev_decision": rev.get("decision", ""),
        "rev_failure_layer": rev.get("failure_layer", ""),
        "rev_reason_codes": rev.get("reason_codes", ""),
        "rev_revision_instructions": rev.get("revision_instructions", ""),
        "rev_parse_ok": rev.get("reviewer_parse_ok", False),
    })
    merged_rows.append(merged)

merged_header = items_header + [
    "rev_decision", "rev_failure_layer",
    "rev_reason_codes", "rev_revision_instructions",
    "rev_parse_ok"
]
with open(ANALYSIS_MERGED_CSV, "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=merged_header)
    w.writeheader()
    for row in merged_rows:
        w.writerow(row)

# sanity stats to console
total = len(items_clean)
gen_ok = sum(1 for r in items_clean if r.get("gen_parse_ok"))
rev_ok = sum(1 for r in reviews_clean if r.get("reviewer_parse_ok"))

# decision counts
decisions_all = {}
for r in reviews_clean:
    decisions_all[r["decision"]] = decisions_all.get(r["decision"], 0) + 1

# decision counts (content only: reviewer JSON parsed OK)
content_reviews = [r for r in reviews_clean if r.get("reviewer_parse_ok")]
decisions_content = {}
for r in content_reviews:
    decisions_content[r["decision"]] = decisions_content.get(r["decision"], 0) + 1

print("\nPOST-RUN CLEANUP SUMMARY")
print(f"- items parsed OK:           {gen_ok}/{total}")
print(f"- reviewer JSON OK:           {rev_ok}/{total}")
print(f"- reviewer parse fails:       {total - rev_ok}")

```

```

print(f"- decision counts (ALL):    {decisions_all}")
print(f"- decision counts (CONTENT):{decisions_content}")

print(f"\nWrote: {ITEMS_CLEAN_CSV}")
print(f"Wrote: {REVIEWS_CLEAN_CSV}")
print(f"Wrote: {ANALYSIS_MERGED_CSV}")

```

3 operations

```

[19]: # load prompts
generator_system = open(os.path.join(PROMPT_DIR, "generator_system.txt"),
    ↪encoding="utf-8").read()
generator_user = open(os.path.join(PROMPT_DIR, "generator_user.txt"),
    ↪encoding="utf-8").read()

reviewer_system = open(os.path.join(PROMPT_DIR, "reviewer_system.txt"),
    ↪encoding="utf-8").read()
reviewer_user_template = open(os.path.join(PROMPT_DIR, "reviewer_user.txt"),
    ↪encoding="utf-8").read()

```

```

[20]: # prep raw output files
write_csv_header_if_needed(GEN_RAW_CSV, ["run_id", "item_id", "gen_output"])
write_csv_header_if_needed(REV_RAW_CSV, ["run_id", "item_id", "review_json"])

print(f"RUN_ID={RUN_ID}")
print(f"Writing RAW generator output to: {GEN_RAW_CSV}")
print(f"Writing RAW reviewer output to: {REV_RAW_CSV}")

```

```

RUN_ID=20260201_125118
Writing RAW generator output to:
C:\Users\kadek\Documents\GitHub\KinaxisCapstone\automated
datasets\generator_output_20260201_125118.csv
Writing RAW reviewer output to:
C:\Users\kadek\Documents\GitHub\KinaxisCapstone\automated
datasets\review_decisions_20260201_125118.csv

```

```

[21]: # run
for i in range(N_ITEMS):
    item_id = f"item_{i+1}"
    print(f"Running {item_id} ({i+1}/{N_ITEMS})")

    # generator (log envelope immediately before request)
    print(
        f"[GENERATOR REQUEST] run_id={RUN_ID} item_id={item_id} "
        f"model={MODEL} temperature={GEN_TEMPERATURE}"
        ↪max_tokens={GEN_MAX_TOKENS} "

```

```

        f"timeout={REQUEST_TIMEOUT_SECONDS}"
    )

    gen_raw = call_llm(
        generator_system,
        generator_user,
        temperature=GEN_TEMPERATURE,
        max_tokens=GEN_MAX_TOKENS
    )

    # clean + enforce generator output contract
    gen_output = clean_generator_text(gen_raw)
    gen_output = hard_trim_after_difficulty(gen_output)

    # write generator output
    with open(GEN_RAW_CSV, "a", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow([RUN_ID, item_id, gen_output])

    time.sleep(SLEEP_SECONDS)

    # pre-review schema gate
    is_valid, violations = validate_generator_schema(gen_output)

    if not is_valid:
        review_norm = {
            "item_id": item_id,
            "decision": "REVISE",
            "failure_layer": "SOFT",
            "reason_codes": ["FORMAT_VIOLATION"],
            "revision_instructions": [
                f"Generator output failed schema gate: {'', '.join(violations)}.
↪"
                "Regenerate item strictly following the required template. "
                "No text after the Difficulty line. Correct Answer must be a
↪single letter A-D."
            ]
        }
        review_json_out = json.dumps(review_norm, ensure_ascii=False)

        with open(REV_RAW_CSV, "a", newline="", encoding="utf-8") as f:
            csv.writer(f).writerow([RUN_ID, item_id, review_json_out])

        continue

    # reviewer: template substitution via replace to avoid brace issues
    reviewer_user = (
        reviewer_user_template

```

```

        .replace("{item_id}", item_id)
        .replace("{exam_item}", gen_output)
    )

    # reviewer (log envelope immediately before request)
    print(
        f"[REVIEWER REQUEST] run_id={RUN_ID} item_id={item_id} "
        f"model={MODEL} temperature={REV_TEMPERATURE}␣
↪max_tokens={REV_MAX_TOKENS} "
        f"timeout={REQUEST_TIMEOUT_SECONDS}"
    )

    review_raw = call_llm(
        reviewer_system,
        reviewer_user,
        temperature=REV_TEMPERATURE,
        max_tokens=REV_MAX_TOKENS
    )

    # extract JSON from reviewer output
    try:
        review_obj = extract_first_json_obj(review_raw)
        review_norm = normalize_decision(item_id, review_obj)

        # post-review hygiene enforcement (deterministic override)
        review_norm = enforce_hygiene_on_review(gen_output, review_norm)

        review_json_out = json.dumps(review_norm, ensure_ascii=False)
    except Exception:
        review_norm = {
            "item_id": item_id,
            "decision": "REJECT",
            "failure_layer": "HARD",
            "reason_codes": ["REVIEWER_PARSE_FAIL"],
            "revision_instructions": []
        }
        review_json_out = json.dumps(review_norm, ensure_ascii=False)

    with open(REV_RAW_CSV, "a", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow([RUN_ID, item_id, review_json_out])

    time.sleep(SLEEP_SECONDS)

print("Batch run complete. Starting post-run cleanup...")
post_run_cleanup(RUN_ID)
print("Done.")

```

Running item_1 (1/5)

```

[GENERATOR REQUEST] run_id=20260201_125118 item_id=item_1 model=deepseek-
coder-6.7b-instruct temperature=0.4 max_tokens=250 timeout=600
[REVIEWER REQUEST] run_id=20260201_125118 item_id=item_1 model=deepseek-
coder-6.7b-instruct temperature=0.0 max_tokens=200 timeout=600
Running item_2 (2/5)
[GENERATOR REQUEST] run_id=20260201_125118 item_id=item_2 model=deepseek-
coder-6.7b-instruct temperature=0.4 max_tokens=250 timeout=600
[REVIEWER REQUEST] run_id=20260201_125118 item_id=item_2 model=deepseek-
coder-6.7b-instruct temperature=0.0 max_tokens=200 timeout=600
Running item_3 (3/5)
[GENERATOR REQUEST] run_id=20260201_125118 item_id=item_3 model=deepseek-
coder-6.7b-instruct temperature=0.4 max_tokens=250 timeout=600
[REVIEWER REQUEST] run_id=20260201_125118 item_id=item_3 model=deepseek-
coder-6.7b-instruct temperature=0.0 max_tokens=200 timeout=600
Running item_4 (4/5)
[GENERATOR REQUEST] run_id=20260201_125118 item_id=item_4 model=deepseek-
coder-6.7b-instruct temperature=0.4 max_tokens=250 timeout=600
[REVIEWER REQUEST] run_id=20260201_125118 item_id=item_4 model=deepseek-
coder-6.7b-instruct temperature=0.0 max_tokens=200 timeout=600
Running item_5 (5/5)
[GENERATOR REQUEST] run_id=20260201_125118 item_id=item_5 model=deepseek-
coder-6.7b-instruct temperature=0.4 max_tokens=250 timeout=600
[REVIEWER REQUEST] run_id=20260201_125118 item_id=item_5 model=deepseek-
coder-6.7b-instruct temperature=0.0 max_tokens=200 timeout=600
Batch run complete. Starting post-run cleanup...

```

POST-RUN CLEANUP SUMMARY

```

- items parsed OK:          3/5
- reviewer JSON OK:        5/5
- reviewer parse fails:    0
- decision counts (ALL):   {'ACCEPT': 3, 'REJECT': 2}
- decision counts (CONTENT):{'ACCEPT': 3, 'REJECT': 2}

```

```

Wrote: C:\Users\kadek\Documents\GitHub\KinaxisCapstone\automated
datasets\items_clean_20260201_125118.csv
Wrote: C:\Users\kadek\Documents\GitHub\KinaxisCapstone\automated
datasets\reviews_clean_20260201_125118.csv
Wrote: C:\Users\kadek\Documents\GitHub\KinaxisCapstone\automated
datasets\analysis_merged_20260201_125118.csv
Done.

```